



University Management System: Phase 2 Report

CSE342: Agile Software Engineering

Submitted to:

- Dr. Mohamed Hassan Mahmoud El Gazzar
- TA. Hazem Zaki
- TA. Abdelrahman Salah

Team Members

- | | | |
|----|-----------------------|---------|
| 1- | Omar Ahmed Abouraia | 23P0100 |
| 2- | Hossam Osama Hussieny | 23P0010 |
| 3- | Mariam Ibrahim Assr | 23P0123 |
| 4- | Mariam Tarek Farouk | 23P0214 |
| 5- | Lujain Mohamed | 23P0187 |

1. Project Overview

This document is the Phase 2 report for the University Management System (UMS), developed using the Scrum framework. Building on the Phase 1 planning artifacts, Phase 2 covers the execution of **two Sprints**, each delivering working increments across four modules: Facilities, Community, Staff, and Curriculum. This report details feature prioritization, MVP backlog items, Scrum event meeting minutes, epic-to-story decomposition, estimation techniques, and the team's Definition of Done.

2. Team Organization & Workspace

The team of 5 manages four modules within a single Jira Scrum project (component-based). GitHub is used for version control.

Member	ID	Module	Role
OmarAhmedAbou Raia	23P0100	Curriculum & Integration	ScrumMaster/ Developer
HossamOsama Hussieny	23P0010	Facilities	Developer
Mariam Ibrahim Assr	23P0123	Staff	ProductOwner/ Developer
Mariam Tarek Farouk	23P0214	Community	Developer
Lujain Mohamed	23P0187	Curriculum	Developer

3. Definition of Done (DoD)

The team reviewed and confirmed the following DoD. An item is "Done" only when ALL conditions below are met:

1. **Format:** Story follows "As a... I want... So that..." structure.
2. **Acceptance Criteria:** At least 3 testable ACs defined in the Jira ticket.
3. **Attributes:** Story is assigned to a Jira Component and linked to its Parent Epic.
4. **Estimation:** Story has a Priority level and Story Point estimate via Planning Poker.
5. **Peer Review:** Reviewed and approved by at least one other team member.
6. **Change Anticipation:** Story description includes a scalability/future-sprint note.
7. **Implementation:** Feature is coded and committed to GitHub; link is in the Jira ticket.
8. **Testing:** All Acceptance Criteria are verified and pass.
9. **Integration Check:** Feature does not break any adjacent module functionality.
10. **Sprint Demo Ready:** Feature can be demonstrated at the Sprint Review.

4. Feature Prioritization

Technique: MoSCoW

The team used **MoSCoW** (Must Have / Should Have / Could Have / Won't Have) to prioritize all features. This was done collaboratively during Sprint Planning. Each feature was scored against three criteria: **Business Value**, **Risk of Delay** (does blocking this block others?), and **Implementation Effort**.

Facilities Module

Category	Feature	Priority	Sprint
Must Have	View Room Availability/ Schedule	High	Sprint 1
Must Have	Manage Student Records	High	Sprint 1
Must Have	Allocate Equipment / Resources	High	Sprint 1
Should Have	Report Maintenance Issues	Medium	Sprint 2
Should Have	Handle Admission Applications	Medium	Sprint 2
Could Have	Booking Notifications	Low	Backlog
Won't Have	AI Resource Optimization	Low	Future

Community Module

Category	Feature	Priority	Sprint
Must Have	Send Message to Professor	High	Sprint 1
Must Have	Reply to Student Messages	High	Sprint 1
Must Have	View Message Inbox	High	Sprint 1
Should Have	Post / View Announcements	Medium	Sprint 2
Could Have	Schedule Meetings	Low	Sprint 2
Could Have	Parent Portal	Low	Backlog
Won't Have	View University Events	Low	Future

Staff Module

Category	Feature	Priority	Sprint
Must Have	View Staff Directory	High	Sprint 1
Must Have	View Assigned Courses	High	Sprint 1
Must Have	Manage TA Responsibilities	High	Sprint 1
Should Have	Maintain Staff Records	High	Sprint 1
Could Have	View Performance Records	Medium	Sprint 2
Could Have	Payroll & Leave Requests	Medium	Sprint 2
Won't Have	Research Publications	Low	Future

Curriculum Module

Category	Feature	Priority	Sprint
Must Have	Semester Course Offering	High	Sprint 1
Must Have	Rule-Based Registration	High	Sprint 1
Must Have	Grade Entry & GPA Calculation	High	Sprint 1
Should Have	Add/Drop & Withdrawal	Medium	Sprint 2
Should Have	Exam Schedule View	Medium	Sprint 2
Could Have	Anonymized Course Surveys	Low	Backlog
Won't Have	Advanced SIS Automation	Low	Future

5. Epic-to-Story Decomposition

Technique Used: User Role Splitting + Functional Path Splitting

Each module started from a large Epic and was broken into smaller, independently deliverable stories following the **INVEST** criteria (Independent, Negotiable, Valuable, Estimable, Small, Testable)

Example: Facilities Epic

Epic: "As a university stakeholder, I want to manage all physical infrastructure."

- **F-01 (Student Path):** "As a Student, I want to view available classrooms so that I can find a study space."
- **F-02 (Admin Path):** "As an Admin, I want to manage the master room schedule so that rooms are assigned without conflicts."
- **F-03 (Admin Path):** "As an Admin, I want to allocate equipment to departments so that university assets are tracked centrally."
- **F-04 (Admin Path):** "As an Admin, I want to maintain student records so that accurate administrative data is always available."

Example: Curriculum Epic

Epic: "As a university member, I want to manage all course-related data."

- **CU-01 (Unit Head Path):** "As a Unit Head, I want to publish the semester's courses so that students can register."
- **CU-02 (Student Path):** "As a Student, I want to register based on GPA rules so that I enroll correctly."
- **CU-03 (Instructor Path):** "As an Instructor, I want to enter grades so that GPA is calculated automatically."
- **CU-04 (Student Path):** "As a Student, I want to add or drop courses so that I can adjust my schedule."

Reasoning: Splitting by user role ensures each story delivers value to a specific actor and can be developed and tested independently. The Unit Head setup path (CU-01) is always developed first because it is a prerequisite for student-facing features.

6. Estimation Technique

Technique: Planning Poker (Story Points using Fibonacci Scale)

The team used **Planning Poker** with the Fibonacci scale (1, 2, 3, 5, 8, 13) for all story estimations. The process was:

1. The Product Owner reads the user story aloud.
2. Each team member privately selects a card representing their effort estimate.
3. All cards are revealed simultaneously to avoid anchoring bias.
4. If estimates differ significantly, team members discuss their reasoning.
5. A second round is played until the team reaches consensus.
6. The agreed estimate is entered as Story Points in Jira.

Reference Baseline

Story Points	Meaning
1	Trivial – simple UI display, no business logic
2	Small – read-only query with one filter
3	Medium – form with validation and basic CRUD
5	Large – feature with multi-role logic and integration
8	Extra Large – cross-module feature with complex rules
13	Too large – must be split before entering Sprint

Sprint 1 Story Point Summary

Story	Module	Points
View Room Availability	Facilities	3
Manage Classroom Schedule	Facilities	5
Allocate Equipment	Facilities	5
Manage Student Records	Facilities	3
Send Message to Professor	Community	3
Reply to Student Messages	Community	3
View Message Inbox	Community	2
View Staff Directory	Staff	2
View Assigned Courses	Staff	3
Manage TA Responsibilities	Staff	5
Maintain Staff Records	Staff	3
Semester Course Offering	Curriculum	5
Rule-Based Registration	Curriculum	8
Grade Entry & GPA Calc	Curriculum	8
Total Sprint 1		58 pts

7. MVP Product Backlog Items

Below are the MVP stories for Sprint 1. Each story is in Jira with ACs, component, epic link, priority, and story points.

Facilities

- **F-01:** "As a Student, I want to view available classrooms so that I can find a study space on campus." (3 pts)
- **F-02:** "As an Admin, I want to manage the master classroom schedule so that rooms are assigned without conflicts." (5 pts)
- **F-03:** "As an Admin, I want to allocate equipment to departments so that university assets are tracked centrally." (5 pts)
- **F-04:** "As an Admin, I want to maintain student records so that accurate data is always available." (3 pts)

Community

- **C-01:** "As a Student, I want to send a message to my professor so that I can seek academic guidance." (3 pts)
- **C-02:** "As a Professor, I want to reply to student messages so that I can provide timely support." (3 pts)
- **C-03:** "As a University Member, I want to view my inbox so that I can manage all communications." (2 pts)

Staff

- **S-01:** "As a Student, I want to view the staff directory so that I can find and contact the right professor." (2 pts)
- **S-02:** "As a Student, I want to see a staff member's assigned courses so that I know who teaches my subject." (3 pts)
- **S-03:** "As an Admin, I want to define TA roles so that TAs have clear duties for labs and grading." (5 pts)
- **S-04:** "As an HR Admin, I want to maintain staff records so that the directory is always current." (3 pts)

Curriculum

- **CU-01:** "As a Unit Head, I want to publish the semester's courses so that students can register." *(5 pts)*
- **CU-02:** "As a Student, I want to register for courses based on GPA rules so that I enroll correctly." *(8 pts)*
- **CU-03:** "As an Instructor, I want to enter grades so that GPA is calculated automatically." *(8 pts)*

8. Sprint 2 Backlog Items

Sprint 2 delivers the "Should Have" features identified in MoSCoW prioritization.

- **F-05:** "As a Staff Member, I want to report a maintenance issue so that facilities are kept in good condition." *(3 pts)*
- **F-06:** "As a Registrar, I want to handle admission applications so that incoming students are processed efficiently." *(8 pts)*
- **C-04:** "As an Admin, I want to post announcements so that all students are informed of university news." *(3 pts)*
- **C-05:** "As a Professor, I want to schedule a meeting with a student so that we can plan sessions in advance." *(5 pts)*
- **S-05:** "As an Admin, I want to view staff performance records so that I can conduct evaluations." *(5 pts)*
- **S-06:** "As a Staff Member, I want to submit leave requests so that HR can manage staffing." *(5 pts)*
- **CU-04:** "As a Student, I want to add or drop a course so that I can adjust my schedule within the allowed period." *(5 pts)*
- **CU-05:** "As a Student, I want to view the exam schedule so that I can plan my preparation." *(3 pts)*

9. Scrum Event Meeting Minutes

9.1 Sprint 1 Planning Meeting

Date: [Insert Date] **Duration:** 2 hours **Attendees:** All 5 team members

Discussion Points:

- Product Owner (Mariam Assr) presented the prioritized Product Backlog.
- The team agreed to target 58 story points for Sprint 1 based on team velocity assessment.
- Planning Poker was conducted for all Sprint 1 stories. Key discussions:
 - Rule-Based Registration (CU-02) and Grade Entry (CU-03) were initially estimated at 5 pts each; after discussion of GPA calculation complexity and multi-role validation logic, consensus moved to 8 pts each.
 - Manage TA Responsibilities (S-03) was discussed as potentially 3 pts but raised to 5 pts after identifying the need to support multiple TAs per course.
- The Sprint Goal was agreed: *"Establish the core academic cycle: rooms are bookable, staff are reachable, students can register, and grades can be recorded."*
- Stories were assigned to team members based on module ownership.

Agreements:

- Sprint duration: 1 week (or as assigned by course).
- Daily Scrums at a fixed time agreed by all members.
- Integration checkpoints scheduled mid-sprint for cross-module dependencies.

9.2 Sprint 1 Daily Scrums (Summary)

Format: Each member answers: (1) What did I do yesterday? (2) What will I do today? (3) Any blockers?

Day 1:

- All members began setting up their module structure and Jira tickets.
- No blockers reported.

Day 2:

- Facilities: Room availability UI in progress.
- Community: Messaging schema designed.
- Staff: Directory API endpoint created.
- Curriculum: Course offering page started.
- **Blocker:** Community module needed the Staff directory API to be ready before embedding the "Send Message" button — resolved by agreeing on a mock API contract.

Day 3:

- Facilities: Schedule management feature complete, awaiting peer review.
- Community: Message send/receive logic implemented.
- Staff: Directory fully functional with search and filter.
- Curriculum: GPA calculation logic under development.
- No new blockers.

Day 4:

- All modules conducting internal testing against Acceptance Criteria.
- Cross-module integration tested: Staff ↔ Community messaging link verified.
- Curriculum ↔ Staff: Instructor assignment pull confirmed working.

Day 5 (Last Day):

- Final fixes and demo preparation.
- All Sprint 1 stories marked Done after AC verification.

9.3 Sprint 1 Review

Date: [Insert Date] **Duration:** 1 hour **Attendees:** All team members (+ Dr. El Gazzar / TAs if applicable)

Features Demonstrated:

- Students can view available classrooms and the weekly schedule.
- Admin can manage the master classroom schedule and allocate equipment.
- Students can send messages to professors; professors can reply; inbox is functional.
- Staff directory is searchable and links directly to the messaging system.
- Unit Head can publish courses; students can register based on GPA rules.
- Instructors can enter grades and GPA is auto-calculated.

Feedback Received:

- The GPA-based registration rule display could be clearer for students — added to Sprint 2 backlog as a UI improvement.
- Request to show room capacity alongside availability — noted for Sprint 2 enhancement.

Sprint 1 Outcome: All 14 stories completed. Sprint Goal achieved.

9.4 Sprint 1 Retrospective

Date: [Insert Date] **Duration:** 45 minutes **Format:** Start / Stop / Continue

Start:

- Begin integration testing earlier (Day 2 instead of Day 4) to catch cross-module issues sooner.
- Add a shared "API Contract" document at Sprint start to prevent mid-sprint blockers.

Stop:

- Waiting for another module to be "100% done" before starting integration — use mock APIs instead.

Continue:

- Planning Poker for estimations — it surfaced hidden complexity effectively.
- Weekly sync meetings for dependency management — very effective.
- Peer review before marking stories as Done — caught 2 issues before demo.

Action Items:

- Omar to create an API Contract doc template for Sprint 2.
- All members to start integration testing from Day 2 in Sprint 2.

9.5 Sprint 2 Planning Meeting

Date: [Insert Date] **Duration:** 1.5 hours **Attendees:** All 5 team members

Discussion Points:

- Sprint 1 velocity was 58 story points. Sprint 2 target set at 37 story points (8 stories, "Should Have" tier).
- Planning Poker conducted for Sprint 2 stories.
- Key discussion: Admission Applications (F-06) estimated at 8 pts due to multi-step workflow (intake, review, decision, notification).
- Sprint Goal agreed: *"Expand the platform with maintenance reporting, announcements, meeting scheduling, performance tracking, leave management, add/drop functionality, and exam scheduling."*

Agreements:

- API Contract document (from Retrospective action item) will be shared on Day 1.
- Integration testing begins Day 2.
- Lujain to own CU-04 (Add/Drop) due to its dependency on CU-02 completed in Sprint 1.

9.6 Sprint 2 Daily Scrums (Summary)

Day 1: Sprint 2 setup, API contract shared. All members started story scaffolding.

Day 2: Integration testing begins. Facilities maintenance form UI done. Community announcements feature in progress.

Day 3: Staff leave request form complete. Curriculum add/drop logic in progress. Exam schedule view designed.

Day 4: All features in testing phase. Announcement "channels" (departmental vs. campus-wide) confirmed working.

Day 5: Final fixes. All stories verified against ACs. Demo preparation complete.

9.7 Sprint 2 Review

Date: [Insert Date] **Duration:** 1 hour

Features Demonstrated:

- Staff can report maintenance issues with category and priority.
- Admins can post announcements; students see them on their dashboard.
- Professors can schedule meetings with students through the platform.
- Admins can view staff performance records.
- Staff can submit leave requests; HR can approve or reject.
- Students can add or drop courses within the allowed window.
- Students can view the full exam schedule with room and time details.

Feedback: Positive overall. Leave request approval flow suggested to include email notification — noted for Phase 3.

Sprint 2 Outcome: All 8 stories completed. Sprint Goal achieved.

9.8 Sprint 2 Retrospective

Start:

- Earlier demo walkthroughs internally (Day 3) to catch UX issues before the final review.

Stop:

- Merging code only at the end of the sprint — merge incrementally to reduce conflicts.

Continue:

- API contracts from Day 1 — this eliminated all mid-sprint integration blockers.
- Planning Poker — consensus-based estimation continues to be reliable.

10. Project Links

- **Jira Scrum Workspace:**
[<https://agileswproject.atlassian.net/jira/software/c/projects/UMS/boards/34/backlog>]
(Contains full Product Backlog, Epic structures, Sprint 1 Backlog, Sprint 2 Backlog, story details, ACs, and story point estimates.)
- **GitHub Repository:** [<https://github.com/OmarAbouraia/University-Management-System.git>]
(Contains Phase 1 documentation, Phase 2 implementation code, and README.)